

Day 19–20: OOP Project

Build:

- Bank Management System
- Library Management System

Part 1 — OOP Project ante enti?

OOP ante **Object-Oriented Programming**.

Real world lo manaki:

- Bank
- Customer
- Account
- Employee
- Book
- Library
- Student
- Vehicle

ilaanti objects untayi.

Java lo ee real-world things ni **classes and objects** ga represent chestham.

Example

Real world:

Bank lo oka customer unnadu.

Customer ki account undi.

Account lo balance undi.

Customer money deposit chesthadu.

Money withdraw chesthadu.

Java lo:

```
class Customer {  
    String name;  
    int customerId;  
}
```

Ikkada Customer oka **class**.

Customer c1 = new Customer();

Ikkada c1 oka **object**.

So:

Real-world entity → Class → Object

Part 2 — Day 19: Bank Management System

First manam Bank Management System build cheddam.

1. Bank Management System lo em untayi?

Oka basic banking application imagine cheyyi.

User ki options:

1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit

Example:

===== BANK MANAGEMENT SYSTEM =====

1. Create Account
2. Deposit
3. Withdraw
4. Check Balance
5. Account Details
6. Exit

User:

Enter choice: 1

Then:

Enter Account Number: 101

Enter Name: Ravi

Enter Initial Balance: 5000

Account create avutundi.

Part 3 — First identify the objects

Project start cheyyadaniki immediate ga code raayakudadhu.

First question:

Mana system lo important objects enti?

Bank system lo:

- Customer
- Account
- Bank
- Transaction

Basic project ki first Account class tho start cheddham.

Part 4 — Account Class

Account ki properties enti?

For example:

Account Number

Customer Name

Balance

So:

```
class Account {
```

```
    int accountNumber;
```

```
    String customerName;
```

```
    double balance;
```

```
}
```

Ikkada:

```
int accountNumber;
```

```
String customerName;
```

```
double balance;
```

ivi **attributes / fields / instance variables**.

Part 5 — Constructor

Account create chesthunappudu values ivvali.

Example:

```
Account a1 = new Account(101, "Ravi", 5000);
```

Daaniki constructor kavali.

```
class Account {
```

```
    int accountNumber;
```

```
    String customerName;
```

```
    double balance;
```

```
    Account(int accountNumber, String customerName, double balance) {
```

```
        this.accountNumber = accountNumber;
```

```
        this.customerName = customerName;
```

```
        this.balance = balance;
```

```
    }
```

```
}
```

this enti?

Suppose:

```
this.accountNumber = accountNumber;
```

Left side:

```
this.accountNumber
```

means object lo unna variable.

Right side:

accountNumber

means constructor ki vachina parameter.

Simple ga:

object variable = input value

Part 6 — Encapsulation apply cheddham

Earlier manam Encapsulation nerchukunnam.

Banking system lo balance ni direct ga modify cheyyakudadhu.

Wrong design:

```
account.balance = -50000;
```

Idi dangerous.

Anduke variables ni private chestham.

```
class Account {
```

```
    private int accountNumber;
```

```
    private String customerName;
```

```
    private double balance;
```

```
}
```

Ippudu outside class nunchi:

```
account.balance
```

direct ga access cheyyalem.

Idi **data hiding**.

Part 7 — Deposit Method

Money deposit cheyyali.

```
public void deposit(double amount) {
```

```
if (amount > 0) {  
    balance = balance + amount;  
    System.out.println("Amount deposited successfully");  
} else {  
    System.out.println("Invalid amount");  
}  
}
```

Suppose balance:

5000

Deposit:

2000

Then:

balance = 5000 + 2000

= 7000

Part 8 — Withdraw Method

Withdraw chesthunappudu rendu conditions important.

Condition 1

Amount positive undali.

Condition 2

Account lo sufficient balance undali.

```
public void withdraw(double amount) {
```

```
    if (amount <= 0) {  
        System.out.println("Invalid amount");  
    }  
    else if (amount > balance) {  
        System.out.println("Insufficient balance");  
    }  
}
```

```
}  
else {  
    balance = balance - amount;  
    System.out.println("Amount withdrawn successfully");  
}  
}
```

Example:

Balance = 7000

Withdraw = 2000

Result:

7000 - 2000 = 5000

But:

Balance = 5000

Withdraw = 8000

Then:

Insufficient balance

Part 9 — Getters

Private variables ni direct ga access cheyyalem.

So getter use chestham.

```
public int getAccountNumber() {  
    return accountNumber;  
}  
  
public String getCustomerName() {  
    return customerName;  
}  
  
public double getBalance() {  
    return balance;  
}
```

Example:

```
System.out.println(account.getBalance());
```

Part 10 — Complete Account Class

Ippudu mana concepts anni combine cheddam.

```
class Account {

    private int accountNumber;
    private String customerName;
    private double balance;

    public Account(int accountNumber, String customerName, double balance) {
        this.accountNumber = accountNumber;
        this.customerName = customerName;
        this.balance = balance;
    }

    public void deposit(double amount) {

        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Amount deposited successfully");
        } else {
            System.out.println("Invalid amount");
        }
    }

    public void withdraw(double amount) {
```

```
if (amount <= 0) {  
    System.out.println("Invalid amount");  
}  
else if (amount > balance) {  
    System.out.println("Insufficient balance");  
}  
else {  
    balance = balance - amount;  
    System.out.println("Amount withdrawn successfully");  
}  
}
```

```
public int getAccountNumber() {  
    return accountNumber;  
}
```

```
public String getCustomerName() {  
    return customerName;  
}
```

```
public double getBalance() {  
    return balance;  
}
```

```
public void displayDetails() {  
  
    System.out.println("Account Number: " + accountNumber);  
    System.out.println("Customer Name: " + customerName);  
    System.out.println("Balance: " + balance);  
}
```

```
}  
}
```

Part 11 — Main Class

Ippudu object create cheddham.

```
public class BankManagementSystem {  
  
    public static void main(String[] args) {  
  
        Account a1 = new Account(101, "Ravi", 5000);  
  
        a1.displayDetails();  
  
        a1.deposit(2000);  
  
        a1.withdraw(1000);  
  
        System.out.println("Final Balance: " + a1.getBalance());  
    }  
}
```

Execution mentally:

Step 1

```
Account a1 = new Account(101, "Ravi", 5000);
```

Object create avutundi.

```
accountNumber = 101
```

```
customerName = Ravi
```

```
balance = 5000
```

Step 2

```
a1.deposit(2000);
```

5000 + 2000 = 7000

Step 3

a1.withdraw(1000);

7000 - 1000 = 6000

Final:

Final Balance: 6000

Part 12 — Multiple Accounts

Real bank lo one account matrame undadu.

Account a1 = new Account(101, "Ravi", 5000);

Account a2 = new Account(102, "Sita", 10000);

Account a3 = new Account(103, "Kiran", 7000);

Ippudu:

a1 → Ravi → 5000

a2 → Sita → 10000

a3 → Kiran → 7000

Each object ki separate data untundi.

Idi **OOP power**.

Part 13 — Array of Objects

Multiple accounts ni store cheyyadaniki array use cheyyachu.

Account[] accounts = new Account[3];

Then:

accounts[0] = new Account(101, "Ravi", 5000);

accounts[1] = new Account(102, "Sita", 10000);

accounts[2] = new Account(103, "Kiran", 7000);

Display:

```
for (int i = 0; i < accounts.length; i++) {  
    accounts[i].displayDetails();  
}
```

Part 14 — Find Account

User account number enter chesthe aa account find cheyyali.

Example:

Enter account number: 102

Manam accounts array lo search chestham.

```
for (int i = 0; i < accounts.length; i++) {  
  
    if (accounts[i].getAccountNumber() == 102) {  
        accounts[i].displayDetails();  
    }  
}
```

Concept:

User input



Account number



Search accounts



Matching account



Perform operation

Part 15 — Scanner

User nunchi input teesukovadaniki:

```
import java.util.Scanner;
```

Then:

```
Scanner sc = new Scanner(System.in);
```

Integer:

```
int choice = sc.nextInt();
```

String:

```
String name = sc.next();
```

Full line:

```
String name = sc.nextLine();
```

Double:

```
double amount = sc.nextDouble();
```

Part 16 — Menu-Driven Bank System

Real project feel ikkada start avutundi.

```
while (true) {
```

```
    System.out.println("1. Create Account");
```

```
    System.out.println("2. Deposit");
```

```
    System.out.println("3. Withdraw");
```

```
    System.out.println("4. Check Balance");
```

```
    System.out.println("5. Display Details");
```

```
    System.out.println("6. Exit");
```

```
    int choice = sc.nextInt();
```

```
    switch (choice) {
```

```
        case 1:
```

```
            // create account
```

```
            break;
```

```
case 2:
    // deposit
    break;

case 3:
    // withdraw
    break;

case 4:
    // check balance
    break;

case 5:
    // display details
    break;

case 6:
    System.out.println("Thank you");
    return;

default:
    System.out.println("Invalid choice");
}
}
```

Why while(true)?

Because menu repeatedly display avvali.

User:

1

operation complete.

Again menu:

1

2

3

4

5

6

User 6 select the exit.

Part 17 — Bank Project OOP Concepts

Mana project lo concepts ela use ayyayo observe cheyyi.

OOP Concept Bank Project lo usage

Class	Account
Object	a1, a2
Constructor	Account creation
Encapsulation	private balance
Methods	deposit(), withdraw()
Array	Multiple accounts
Abstraction	Banking operations hide cheyyadam
Polymorphism	Different account types
Inheritance	SavingsAccount / CurrentAccount
Interface	Transaction rules

Part 18 — Inheritance add cheddham

Suppose bank lo different account types unnayi.

Account

↓

SavingsAccount

↓

CurrentAccount

Common things:

accountNumber

customerName

balance

deposit()

withdraw()

So parent class:

```
class Account {
```

```
    protected int accountNumber;
```

```
    protected String customerName;
```

```
    protected double balance;
```

```
    public void deposit(double amount) {
```

```
        balance += amount;
```

```
    }
```

```
    public void withdraw(double amount) {
```

```
        if (amount <= balance) {
```

```
            balance -= amount;
```

```
        }
```

```
    }
```

```
}
```

Child:

```
class SavingsAccount extends Account {
```

```
}
```

Current:

```
class CurrentAccount extends Account {
```

```
}
```

extends means:

SavingsAccount is an Account.

Part 19 — Polymorphism in Bank

Suppose:

Account account;

It can refer to:

SavingsAccount

or

CurrentAccount

Example:

```
Account a = new SavingsAccount();
```

This is runtime polymorphism.

If both classes have their own:

```
withdraw()
```

implementation, Java runtime lo actual object based on method decide chestundi.

Part 20 — Interface in Bank

Suppose every bank account must follow transaction rules.

```
interface Transaction {
```

```
    void deposit(double amount);
```

```
    void withdraw(double amount);
```

```
}
```

Then:

```
class SavingsAccount implements Transaction {
```

```
    public void deposit(double amount) {
```

```
        // implementation
```

```
    }
```

```
    public void withdraw(double amount) {
```

```
        // implementation
```

```
    }
```

```
}
```

Meaning:

"Transaction interface cheppina rules ni SavingsAccount follow cheyyali."

Part 21 — Bank Project Architecture

Large project lo classes separate ga untayi.

BankManagementSystem

```
|
```

```
|---- Account
```

```
|
```

```
|---- SavingsAccount
```

```
|
```

```
|---- CurrentAccount
```

```
|
```

```
|---- Customer
```

```
|
```

```
|---- Transaction
```

```
|
```

|---- Bank

Idi professional project design ki first step.

Part 22 — Day 20: Library Management System

Ippudu second project.

Library lo:

Books

Students/Members

Librarian

Issue

Return

Search

untayi.

Basic menu:

===== LIBRARY MANAGEMENT SYSTEM =====

1. Add Book
 2. Display Books
 3. Search Book
 4. Issue Book
 5. Return Book
 6. Exit
-

Part 23 — Identify Classes

First real-world entities identify cheddam.

Book

Member

Library

Optional:

Librarian

Transaction

Basic project ki:

Book

Member

Library

chaalu.

Part 24 — Book Class

Book ki information:

Book ID

Title

Author

Availability

So:

```
class Book {
```

```
    private int bookId;
```

```
    private String title;
```

```
    private String author;
```

```
    private boolean available;
```

```
}
```

boolean only two values:

true

false

Here:

```
available = true;
```

means book available.

available = false;
means book issued.

Part 25 — Book Constructor

```
public Book(int bookId, String title, String author) {  
  
    this.bookId = bookId;  
    this.title = title;  
    this.author = author;  
    this.available = true;  
}
```

New book create chesthe automatically:

available = true

Part 26 — Display Book

```
public void displayBook() {  
  
    System.out.println("Book ID: " + bookId);  
    System.out.println("Title: " + title);  
    System.out.println("Author: " + author);  
    System.out.println("Available: " + available);  
}
```

Part 27 — Issue Book

Book issue chesthe:

```
public void issueBook() {  
  
    if (available) {  
        available = false;
```

```
        System.out.println("Book issued successfully");
    } else {
        System.out.println("Book is already issued");
    }
}
```

Logic:

available?

```
|
|--- YES → issue → false
|
|--- NO → already issued
```

Part 28 — Return Book

```
public void returnBook() {

    if (!available) {
        available = true;
        System.out.println("Book returned successfully");
    } else {
        System.out.println("Book was not issued");
    }
}
```

Part 29 — Complete Book Class

```
class Book {

    private int bookId;
    private String title;
    private String author;
```

```
private boolean available;
```

```
public Book(int bookId, String title, String author) {
```

```
    this.bookId = bookId;
```

```
    this.title = title;
```

```
    this.author = author;
```

```
    this.available = true;
```

```
}
```

```
public void issueBook() {
```

```
    if (available) {
```

```
        available = false;
```

```
        System.out.println("Book issued successfully");
```

```
    } else {
```

```
        System.out.println("Book is already issued");
```

```
    }
```

```
}
```

```
public void returnBook() {
```

```
    if (!available) {
```

```
        available = true;
```

```
        System.out.println("Book returned successfully");
```

```
    } else {
```

```
        System.out.println("Book was not issued");
```

```
    }
```

```
}
```

```

public void displayBook() {

    System.out.println("Book ID: " + bookId);
    System.out.println("Title: " + title);
    System.out.println("Author: " + author);
    System.out.println("Available: " + available);
}

public int getBookId() {
    return bookId;
}
}

```

Part 30 — Multiple Books

```
Book[] books = new Book[3];
```

```

books[0] = new Book(1, "Java Basics", "James");
books[1] = new Book(2, "Python", "Guido");
books[2] = new Book(3, "Data Structures", "Mark");

```

Memory concept roughly:

books

```

|
+----> Book object 1
|
+----> Book object 2
|
+----> Book object 3

```

Array stores references to objects.

Part 31 — Search Book

User:

Enter Book ID: 2

Search:

```
for (int i = 0; i < books.length; i++) {
```

```
    if (books[i].getBookId() == 2) {
```

```
        books[i].displayBook();
```

```
        break;
```

```
    }
```

```
}
```

Logic:

Book 1 → ID 1 → not match

Book 2 → ID 2 → MATCH

Book 3 → not checked

break loop ni stop chestundi.

Part 32 — Member Class

Library lo book issue cheyyadaniki member undali.

```
class Member {
```

```
    private int memberId;
```

```
    private String name;
```

```
    public Member(int memberId, String name) {
```

```
        this.memberId = memberId;
        this.name = name;
    }

    public void displayMember() {

        System.out.println("Member ID: " + memberId);
        System.out.println("Name: " + name);
    }
}
```

Part 33 — Library Class

Ippudu main important class:

```
class Library {

    private Book[] books;

}
```

Library has books.

Idi **HAS-A relationship**.

Library HAS-A Book

Actually multiple books:

Library HAS-A collection of Books

Part 34 — Composition / Aggregation Idea

OOP lo relationships important.

IS-A

SavingsAccount IS-A Account

Inheritance.

```
class SavingsAccount extends Account
```

HAS-A

Library HAS-A Book

Meaning Library object to Book objects maintain chesthundi.

Example:

```
class Library {
```

```
    Book[] books;
```

```
}
```

Part 35 — Complete Simple Library Project

```
import java.util.Scanner;
```

```
class Book {
```

```
    private int bookId;
```

```
    private String title;
```

```
    private String author;
```

```
    private boolean available;
```

```
    public Book(int bookId, String title, String author) {
```

```
        this.bookId = bookId;
```

```
        this.title = title;
```

```
        this.author = author;
```

```
        this.available = true;
```

```
}
```

```
public int getBookId() {  
    return bookId;  
}
```

```
public void displayBook() {  
  
    System.out.println("Book ID: " + bookId);  
    System.out.println("Title: " + title);  
    System.out.println("Author: " + author);  
    System.out.println("Available: " + available);  
}
```

```
public void issueBook() {  
  
    if (available) {  
        available = false;  
        System.out.println("Book issued successfully");  
    } else {  
        System.out.println("Book is already issued");  
    }  
}
```

```
public void returnBook() {  
  
    if (!available) {  
        available = true;  
        System.out.println("Book returned successfully");  
    } else {  
        System.out.println("Book was not issued");  
    }  
}
```

```
    }  
    }  
}
```

```
public class LibraryManagementSystem {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        Book[] books = new Book[3];
```

```
        books[0] = new Book(1, "Java Basics", "James");
```

```
        books[1] = new Book(2, "Python", "Guido");
```

```
        books[2] = new Book(3, "Data Structures", "Mark");
```

```
        while (true) {
```

```
            System.out.println("\n===== LIBRARY MANAGEMENT SYSTEM =====");
```

```
            System.out.println("1. Display Books");
```

```
            System.out.println("2. Search Book");
```

```
            System.out.println("3. Issue Book");
```

```
            System.out.println("4. Return Book");
```

```
            System.out.println("5. Exit");
```

```
            System.out.print("Enter choice: ");
```

```
            int choice = sc.nextInt();
```

```
            switch (choice) {
```

case 1:

```
for (int i = 0; i < books.length; i++) {  
    books[i].displayBook();  
    System.out.println();  
}
```

break;

case 2:

```
System.out.print("Enter Book ID: ");  
int searchId = sc.nextInt();
```

```
for (int i = 0; i < books.length; i++) {  
  
    if (books[i].getBookId() == searchId) {  
  
        books[i].displayBook();  
        break;  
    }  
}
```

break;

case 3:

```
System.out.print("Enter Book ID: ");
```

```
int issuelId = sc.nextInt();

for (int i = 0; i < books.length; i++) {

    if (books[i].getBookId() == issuelId) {

        books[i].issueBook();
        break;
    }
}

break;
```

case 4:

```
System.out.print("Enter Book ID: ");
int returnId = sc.nextInt();

for (int i = 0; i < books.length; i++) {

    if (books[i].getBookId() == returnId) {

        books[i].returnBook();
        break;
    }
}

break;
```

case 5:

```
    System.out.println("Thank you!");  
    return;
```

default:

```
    System.out.println("Invalid choice");  
}  
}  
}
```

Part 36 — Ee project ni line-by-line ela think cheyyali?

Important point.

Code memorize cheyyakudadhu.

Requirement → Classes → Variables → Methods → Objects → Menu

Ee order follow cheyyali.

Step 1 — Requirement

"Library lo books manage cheyyali."

Step 2 — Objects

Book

Member

Library

Step 3 — Book properties

id

title

author

availability

Step 4 — Operations

display

search

issue

return

Step 5 — Class

class Book

Step 6 — Objects

Book b1 = new Book(...);

Step 7 — Multiple objects

Book[] books

Step 8 — User interaction

Scanner

Step 9 — Menu

switch

Step 10 — Repeat

while

Idi project-building thinking.

Part 37 — Bank vs Library

Bank	Library
Account	Book
Customer	Member
Balance	Availability
Deposit	Issue
Withdraw	Return
Account search	Book search
Account details	Book details

Bank Library

Bank Library

Notice cheyyi:

Same OOP principles different real-world problems ki apply avuthunnayi.

Part 38 — Encapsulation ekkada use ayyindi?

Bank:

private double balance;

Library:

private boolean available;

Why?

Because outside code direct ga change cheyyakudadhu.

Bank:

balance = -100000;

Library:

available = true;

ila random ga modify cheyyakunda methods dwara control chestham.

deposit()

withdraw()

issueBook()

returnBook()

Idi Encapsulation.

Part 39 — Abstraction ekkada?

User ki:

Deposit

Withdraw

ane operation kanipisthundi.

But internally:

amount validation

balance checking

balance update

ela jarugutundo user ki teliyalsina avasaram ledu.

Similarly library user:

Issue Book

click chesthadu.

Internal logic:

book available?

↓

yes/no

↓

availability update

hide chestham.

Relevant internal implementation hide chesi required functionality matrame expose cheyyadam = Abstraction.

Part 40 — Inheritance ekkada useful?

Bank lo:

Account

|

+--- SavingsAccount

|

+--- CurrentAccount

Library lo:

User

|

+--- StudentMember

|

+--- FacultyMember

|

+--- Librarian

Common properties parent class lo pettochu.

Example:

```
class User {
```

```
    String name;
```

```
    int id;
```

```
}
```

Then:

```
class StudentMember extends User {
```

```
}
```

Part 41 — Polymorphism ekkada useful?

Suppose:

User u;

u can refer to:

StudentMember

or

FacultyMember

Each type ki different borrowing limit undachu.

```
class StudentMember extends User {
```

```
    void borrowBook() {
```

```
        System.out.println("Student borrowing rules");
```

```
    }
```

```
}
```

```

class FacultyMember extends User {

    void borrowBook() {
        System.out.println("Faculty borrowing rules");
    }
}

```

Then:

```

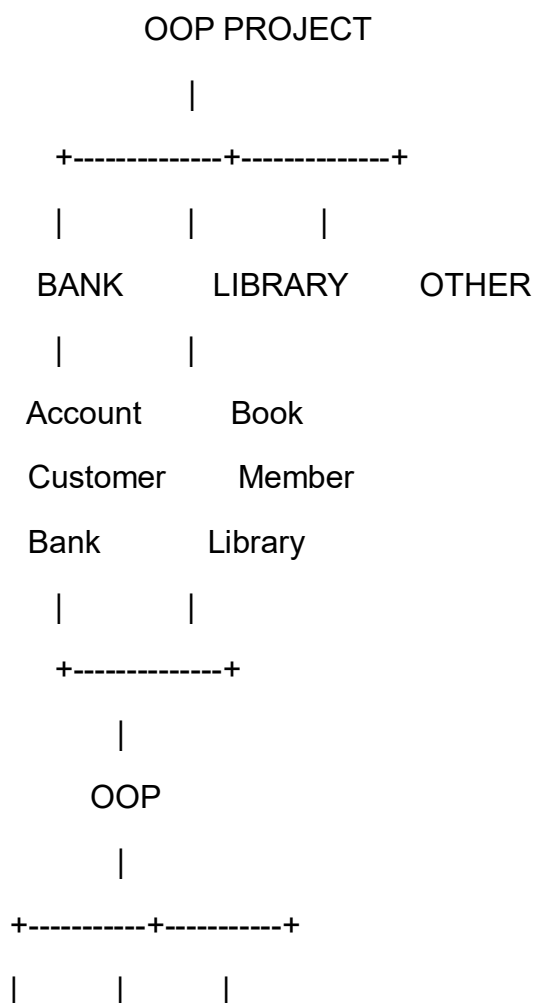
User u = new StudentMember();
u.borrowBook();

```

Actual object StudentMember, so its overridden method execute avutundi.

Part 42 — Project lo OOP complete picture

Remember this diagram:



Class/Object Encapsulation Inheritance

| | |

Constructor Data Hiding IS-A

|

Methods

|

Polymorphism

|

Abstraction

Part 43 — Important terminology

Project to see words frequently in tav.

Entity

Real-world thing.

Book

Account

Customer

Attribute

Entity property.

Book → title

Book → author

Account → balance

Method

Entity action.

deposit()

withdraw()

issueBook()

returnBook()

Object

Class nunchi create chesina actual instance.

Account a1 = new Account(...);

State

Object current data.

Example:

Account balance = ₹5000

Behavior

Object perform chese action.

deposit()

withdraw()

Part 44 — Beginner mistakes

Mistake 1: Everything in main()

Wrong approach:

```
main() {  
    500 lines of code  
}
```

Better:

Account class

Customer class

Bank class

Separate responsibilities.

Mistake 2: Public variables

Avoid:

```
public double balance;
```

Prefer:

```
private double balance;
```

Mistake 3: Code without validation

Wrong:

```
balance -= amount;
```

Correct:

```
if (amount <= balance) {  
    balance -= amount;  
}
```

Mistake 4: Memorizing project

Project ni memorize cheyyaku.

Instead ask:

"Ee real-world system lo entities enti?"

Then automatically classes identify cheyyagalav.

Part 45 — How to build ANY OOP project

Ee formula ni remember chesko:

1. Understand problem

↓

2. Identify real-world entities

↓

3. Convert entities into classes

↓

4. Identify attributes

↓

5. Identify behaviors

↓

6. Create constructors

↓

7. Apply encapsulation



8. Create objects



9. Connect objects



10. Add menu/input



11. Add validation



12. Test

Idi **very important**.

Future lo:

Student Management System

Employee Management System

Hospital Management System

Vehicle Rental System

Hotel Management System

anything ichina same process use cheyyachu.

Part 46 — Day 19–20 Practice Tasks

First **Bank Management System** lo nuvvu yourself try cheyyali:

Level 1

Create:

Account

with:

accountNumber

name

balance

Methods:

deposit()

withdraw()

display()

Level 2

Add:

Scanner

switch

while

Level 3

Add multiple accounts:

Account[]

Level 4

Add:

Search account

Level 5

Add:

SavingsAccount

CurrentAccount

using inheritance.

Library Project Practice

Level 1

Create:

Book

Level 2

Add:

issueBook()

returnBook()

Level 3

Add:

Book[]

Level 4

Add:

Search

Level 5

Create:

Member

Library

Level 6

Add inheritance:

Member

↓

StudentMember

FacultyMember

OOP concepts ni real-world problem ki convert cheyyadam nerchukovadam.

You should be able to see:

REAL WORLD

↓

ENTITIES

↓

CLASSES

↓

OBJECTS

↓

ATTRIBUTES

↓

METHODS



RELATIONSHIPS



OOP CONCEPTS



WORKING APPLICATION

For Bank:

Bank

├── Customer

└── Account

 ├── account number

 ├── name

 ├── balance

 ├── deposit()

 └── withdraw()

For Library:

Library

├── Books

| ├── id

| ├── title

| ├── author

| ├── issueBook()

| └── returnBook()

|

└── Members

 ├── id

 └── name

One golden rule: OOP project lo first code gurinchi think cheyyaku. **First "real world lo em objects unnayi?" ani think cheyyi.** Classes, objects, variables, methods anni dani nunchi naturally derive avuthayi.

1. Bank Management System — Complete Code

Save the file as:

BankManagementSystem.java

```
import java.util.Scanner;
```

```
// ===== ACCOUNT CLASS =====
```

```
class Account {
```

```
    private int accountNumber;
```

```
    private String customerName;
```

```
    private double balance;
```

```
    // Constructor
```

```
    public Account(int accountNumber, String customerName, double balance) {
```

```
        this.accountNumber = accountNumber;
```

```
        this.customerName = customerName;
```

```
        this.balance = balance;
```

```
    }
```

```
    // Deposit money
```

```
    public void deposit(double amount) {
```

```
        if (amount <= 0) {
```

```
            System.out.println("Invalid deposit amount.");
```

```
        } else {
```

```
        balance = balance + amount;

        System.out.println("Amount deposited successfully.");
    }
}
```

// Withdraw money

```
public void withdraw(double amount) {

    if (amount <= 0) {
        System.out.println("Invalid withdrawal amount.");
    }
    else if (amount > balance) {
        System.out.println("Insufficient balance.");
    }
    else {
        balance = balance - amount;
        System.out.println("Amount withdrawn successfully.");
    }
}
```

// Get account number

```
public int getAccountNumber() {
    return accountNumber;
}
```

// Get customer name

```
public String getCustomerName() {
    return customerName;
}
```

```

// Get balance
public double getBalance() {
    return balance;
}

// Display account details
public void displayDetails() {

    System.out.println("-----");
    System.out.println("Account Number : " + accountNumber);
    System.out.println("Customer Name : " + customerName);
    System.out.println("Balance      : " + balance);
    System.out.println("-----");
}
}

```

```

// ===== MAIN CLASS =====

```

```

public class BankManagementSystem {

    static Scanner sc = new Scanner(System.in);

    // Maximum number of accounts
    static Account[] accounts = new Account[100];

    // Current number of accounts
    static int accountCount = 0;

```

```
// ===== CREATE ACCOUNT =====
```

```
public static void createAccount() {
```

```
    System.out.println("\n===== CREATE ACCOUNT =====");
```

```
    System.out.print("Enter Account Number: ");
```

```
    int accountNumber = sc.nextInt();
```

```
    // Check duplicate account number
```

```
    if (findAccount(accountNumber) != null) {
```

```
        System.out.println("Account number already exists.");
```

```
        return;
```

```
    }
```

```
    sc.nextLine();
```

```
    System.out.print("Enter Customer Name: ");
```

```
    String name = sc.nextLine();
```

```
    System.out.print("Enter Initial Balance: ");
```

```
    double balance = sc.nextDouble();
```

```
    if (balance < 0) {
```

```
        System.out.println("Balance cannot be negative.");
```

```
        return;
```

```

    }

    Account account =
        new Account(accountNumber, name, balance);

    accounts[accountCount] = account;

    accountCount++;

    System.out.println("Account created successfully.");
}

// ===== FIND ACCOUNT =====

public static Account findAccount(int accountNumber) {

    for (int i = 0; i < accountCount; i++) {

        if (accounts[i].getAccountNumber() == accountNumber) {

            return accounts[i];
        }
    }

    return null;
}

```

```
// ===== DEPOSIT =====
```

```
public static void depositMoney() {
```

```
    System.out.println("\n===== DEPOSIT MONEY =====");
```

```
    System.out.print("Enter Account Number: ");
```

```
    int accountNumber = sc.nextInt();
```

```
    Account account = findAccount(accountNumber);
```

```
    if (account == null) {
```

```
        System.out.println("Account not found.");
```

```
        return;
```

```
    }
```

```
    System.out.print("Enter Amount: ");
```

```
    double amount = sc.nextDouble();
```

```
    account.deposit(amount);
```

```
}
```

```
// ===== WITHDRAW =====
```

```
public static void withdrawMoney() {
```

```
    System.out.println("\n===== WITHDRAW MONEY =====");
```

```

System.out.print("Enter Account Number: ");
int accountNumber = sc.nextInt();

Account account = findAccount(accountNumber);

if (account == null) {

    System.out.println("Account not found.");
    return;
}

System.out.print("Enter Amount: ");
double amount = sc.nextDouble();

account.withdraw(amount);
}

// ===== CHECK BALANCE =====

public static void checkBalance() {

    System.out.println("\n===== CHECK BALANCE =====");

    System.out.print("Enter Account Number: ");
    int accountNumber = sc.nextInt();

    Account account = findAccount(accountNumber);

```

```

    if (account == null) {

        System.out.println("Account not found.");
        return;
    }

    System.out.println(
        "Current Balance: " + account.getBalance()
    );
}

// ===== DISPLAY ACCOUNT =====

public static void displayAccount() {

    System.out.println("\n===== ACCOUNT DETAILS =====");

    System.out.print("Enter Account Number: ");
    int accountNumber = sc.nextInt();

    Account account = findAccount(accountNumber);

    if (account == null) {

        System.out.println("Account not found.");
        return;
    }
}

```

```

        account.displayDetails();
    }

// ===== DISPLAY ALL ACCOUNTS =====

public static void displayAllAccounts() {

    System.out.println("\n===== ALL ACCOUNTS =====");

    if (accountCount == 0) {

        System.out.println("No accounts available.");
        return;
    }

    for (int i = 0; i < accountCount; i++) {

        accounts[i].displayDetails();
    }
}

// ===== MAIN METHOD =====

public static void main(String[] args) {

    while (true) {

```

```
System.out.println("\n=====");
System.out.println("    BANK MANAGEMENT SYSTEM");
System.out.println("=====");
```

```
System.out.println("1. Create Account");
System.out.println("2. Deposit Money");
System.out.println("3. Withdraw Money");
System.out.println("4. Check Balance");
System.out.println("5. Display Account");
System.out.println("6. Display All Accounts");
System.out.println("7. Exit");
```

```
System.out.print("Enter your choice: ");
```

```
int choice = sc.nextInt();
```

```
switch (choice) {
```

```
    case 1:
        createAccount();
        break;
```

```
    case 2:
        depositMoney();
        break;
```

```
    case 3:
        withdrawMoney();
```

```
        break;

    case 4:
        checkBalance();
        break;

    case 5:
        displayAccount();
        break;

    case 6:
        displayAllAccounts();
        break;

    case 7:
        System.out.println(
            "Thank you for using Bank Management System."
        );
        sc.close();
        return;

    default:
        System.out.println("Invalid choice.");
    }
}
}
```

How Bank Code Works

Overall flow:

main()



Menu



User chooses option



switch



createAccount()

depositMoney()

withdrawMoney()

checkBalance()

displayAccount()



findAccount()



Account object



Account method

Example

If user chooses:

2. Deposit Money

Flow:

depositMoney()



Account Number



findAccount()

↓

Account object

↓

account.deposit(amount)

↓

balance updated

OOP Concepts Used

Encapsulation

```
private int accountNumber;
```

```
private String customerName;
```

```
private double balance;
```

Data directly access cheyyakunda methods use chestunnam.

Constructor

```
public Account(int accountNumber,  
                String customerName,  
                double balance)
```

Object create chestunnappudu initial values set chestundi.

Object

```
Account account =  
    new Account(accountNumber, name, balance);
```

Array of Objects

```
static Account[] accounts = new Account[100];  
100 account references store cheyyagalamu.
```

Method

```
deposit()
```

```
withdraw()
```

```
displayDetails()
```

Search

findAccount()

2. Library Management System — Complete Code

Save this as:

LibraryManagementSystem.java

```
import java.util.Scanner;
```

```
// ===== BOOK CLASS =====
```

```
class Book {
```

```
    private int bookId;
```

```
    private String title;
```

```
    private String author;
```

```
    private boolean available;
```

```
    // Constructor
```

```
    public Book(int bookId, String title, String author) {
```

```
        this.bookId = bookId;
```

```
        this.title = title;
```

```
        this.author = author;
```

```
        // New book is available
```

```
        this.available = true;
```

```
    }
```

```
// Get Book ID
```

```
public int getBookId() {  
    return bookId;  
}
```

```
// Get availability
```

```
public boolean isAvailable() {  
    return available;  
}
```

```
// Issue Book
```

```
public void issueBook() {
```

```
    if (available) {
```

```
        available = false;
```

```
        System.out.println(  
            "Book issued successfully."  
        );
```

```
    } else {
```

```
        System.out.println(  
            "Book is already issued."  
        );
```

```
}  
}
```

```
// Return Book
```

```
public void returnBook() {
```

```
    if (!available) {
```

```
        available = true;
```

```
        System.out.println(
```

```
            "Book returned successfully."
```

```
        );
```

```
    } else {
```

```
        System.out.println(
```

```
            "This book is already available."
```

```
        );
```

```
    }
```

```
}
```

```
// Display Book Details
```

```
public void displayBook() {
```

```
    System.out.println("-----");
```

```

        System.out.println(
            "Book ID    : " + bookId
        );

        System.out.println(
            "Title      : " + title
        );

        System.out.println(
            "Author     : " + author
        );

        System.out.println(
            "Availability : "
            + (available ? "Available" : "Issued")
        );

        System.out.println("-----");
    }
}

// ===== MEMBER CLASS =====

class Member {

    private int memberId;
    private String name;

```

// Constructor

```
public Member(int memberId, String name) {
```

```
    this.memberId = memberId;
```

```
    this.name = name;
```

```
}
```

// Get Member ID

```
public int getMemberId() {
```

```
    return memberId;
```

```
}
```

// Display Member

```
public void displayMember() {
```

```
    System.out.println("-----");
```

```
    System.out.println(
```

```
        "Member ID : " + memberId
```

```
    );
```

```
    System.out.println(
```

```
        "Name      : " + name
```

```
    );
```

```
    System.out.println("-----");
```

```

    }
}

// ===== MAIN CLASS =====

public class LibraryManagementSystem {

    static Scanner sc = new Scanner(System.in);

    // Store books
    static Book[] books = new Book[100];

    // Number of books
    static int bookCount = 0;

    // Store members
    static Member[] members = new Member[100];

    // Number of members
    static int memberCount = 0;

    // ===== ADD BOOK =====

    public static void addBook() {

```

```
System.out.println("\n===== ADD BOOK =====");
```

```
System.out.print("Enter Book ID: ");
```

```
int bookId = sc.nextInt();
```

```
// Check duplicate
```

```
if (findBook(bookId) != null) {
```

```
    System.out.println(
```

```
        "Book ID already exists."
```

```
    );
```

```
    return;
```

```
}
```

```
sc.nextLine();
```

```
System.out.print("Enter Book Title: ");
```

```
String title = sc.nextLine();
```

```
System.out.print("Enter Author Name: ");
```

```
String author = sc.nextLine();
```

```
Book book =
```

```
    new Book(bookId, title, author);
```

```
books[bookCount] = book;
```

```
bookCount++;

System.out.println(
    "Book added successfully."
);
}

// ===== FIND BOOK =====

public static Book findBook(int bookId) {

    for (int i = 0; i < bookCount; i++) {

        if (books[i].getBookId() == bookId) {

            return books[i];
        }
    }

    return null;
}

// ===== DISPLAY BOOKS =====

public static void displayBooks() {
```

```

System.out.println("\n===== ALL BOOKS =====");

if (bookCount == 0) {

    System.out.println(
        "No books available."
    );

    return;
}

for (int i = 0; i < bookCount; i++) {

    books[i].displayBook();
}

}

// ===== SEARCH BOOK =====

public static void searchBook() {

    System.out.println("\n===== SEARCH BOOK =====");

    System.out.print("Enter Book ID: ");
    int bookId = sc.nextInt();

```

```

Book book = findBook(bookId);

if (book == null) {

    System.out.println(
        "Book not found."
    );

} else {

    book.displayBook();
}
}

// ===== ISSUE BOOK =====

public static void issueBook() {

    System.out.println("\n===== ISSUE BOOK =====");

    System.out.print("Enter Book ID: ");
    int bookId = sc.nextInt();

    Book book = findBook(bookId);

```

```

        if (book == null) {

            System.out.println(
                "Book not found."
            );

            return;
        }

        book.issueBook();
    }

    // ===== RETURN BOOK =====

    public static void returnBook() {

        System.out.println("\n===== RETURN BOOK =====");

        System.out.print("Enter Book ID: ");
        int bookId = sc.nextInt();

        Book book = findBook(bookId);

        if (book == null) {

```

```

        System.out.println(
            "Book not found."
        );

        return;
    }

    book.returnBook();
}

// ===== ADD MEMBER =====

public static void addMember() {

    System.out.println("\n===== ADD MEMBER =====");

    System.out.print("Enter Member ID: ");
    int memberId = sc.nextInt();

    sc.nextLine();

    System.out.print("Enter Member Name: ");
    String name = sc.nextLine();

```

```

Member member =
    new Member(memberId, name);

members[memberCount] = member;

memberCount++;

System.out.println(
    "Member added successfully."
);
}

// ===== DISPLAY MEMBERS =====

public static void displayMembers() {

    System.out.println("\n===== ALL MEMBERS =====");

    if (memberCount == 0) {

        System.out.println(
            "No members available."
        );

        return;
    }

```

```
        for (int i = 0; i < memberCount; i++) {

            members[i].displayMember();

        }
    }
}
```

```
// ===== MAIN =====
```

```
public static void main(String[] args) {
```

```
    while (true) {
```

```
        System.out.println("\n=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
```

```
        System.out.println("1. Add Book");
        System.out.println("2. Display Books");
        System.out.println("3. Search Book");
        System.out.println("4. Issue Book");
        System.out.println("5. Return Book");
        System.out.println("6. Add Member");
        System.out.println("7. Display Members");
        System.out.println("8. Exit");
```

```
System.out.print("Enter your choice: ");
```

```
int choice = sc.nextInt();
```

```
switch (choice) {
```

```
    case 1:
```

```
        addBook();
```

```
        break;
```

```
    case 2:
```

```
        displayBooks();
```

```
        break;
```

```
    case 3:
```

```
        searchBook();
```

```
        break;
```

```
    case 4:
```

```
        issueBook();
```

```
        break;
```

case 5:

returnBook();

break;

case 6:

addMember();

break;

case 7:

displayMembers();

break;

case 8:

System.out.println(

"Thank you for using Library Management System."

);

sc.close();

return;

default:

System.out.println(

```
                "Invalid choice."
            );
        }
    }
}
```

Library Project Flow

Suppose user chooses:

1. Add Book

Flow:

main()



switch



addBook()



Book object created



books[] array to store



bookCount++

If user chooses:

4. Issue Book

Flow:

issueBook()



Book ID



findBook()



Book object



book.issueBook()



available = false

Return:

returnBook()



findBook()



book.returnBook()



available = true

Most Important Difference Between the Two

Bank

Main object:

Account

Account has:

accountNumber

customerName

balance

Operations:

deposit()

withdraw()

Library

Main object:

Book

Book has:

bookId

title

author

available

Operations:

issueBook()

returnBook()

How to Run

Create **two separate .java files**.

File 1

BankManagementSystem.java

Run:

```
javac BankManagementSystem.java
```

```
java BankManagementSystem
```

File 2

LibraryManagementSystem.java

Run:

```
javac LibraryManagementSystem.java
```

```
java LibraryManagementSystem
```

Or VS Code lo respective file open chesi **Run Java** click cheyyachu.

One thing you should understand before modifying these projects

For both projects, **main() is not the actual OOP logic**.

For Bank:

BankManagementSystem

↓

controls



Account



deposit / withdraw / balance

For Library:

LibraryManagementSystem



controls



Book



issue / return / availability

So project ni understand cheyyadaniki **first Account and Book classes ni study cheyyi**, then main() ki vellali. This order lo chusthe OOP project structure clear ga ardham avutundi.